



电子科技大学

University of Electronic Science and Technology of China

信息与软件工程学院

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

W1D5:Linux操作系统并发编程之 多线程

Linux下Web Server的设计

讲授内容

- 训练目的
- 训练原理
- 训练内容



- 训练目的

- 掌握UNIX/Linux系统创建线程的方法
- 理解线程启动例程的设计思想
- 掌握线程终止的方式
- 掌握线程之间共享数据的方法
- 掌握等待线程终止的方法



- 训练目的：
 - 了解临界资源、哲学家进餐问题、死锁等经典的操作系统同步互斥问题的基本概念
 - 掌握利用互斥量机制实现多线程对临界资源互斥访问的方法
 - 掌握互斥量非阻塞加锁的操作方法
 - 掌握在保证对临界资源互斥访问的基础上通过让权等待的方式预防死锁的编程思想



训练原理：线程的创建

- pthread_create函数用于创建一个线程
- 函数原型
 - 头文件：pthread.h
 - **int pthread_create**(pthread_t *restrict tidp,
 const pthread_attr_t *restrict attr,
 void *(*start_rtn)(void *),
 void *restrict arg);
- 调用pthread_create函数的线程是所创建线程的父线程



训练原理：线程的创建

- 参数

- tidp: 指向线程ID的指针，当函数成功返回时将存储所创建的子线程ID
- attr: 用于指定所创建线程的属性（一般直接传入空指针NULL，表示线程属性采用默认值）
- start_rtn: 线程的启动例程函数指针，新创建的线程入口点是该函数
- arg: 向线程的启动例程函数传递信息的参数

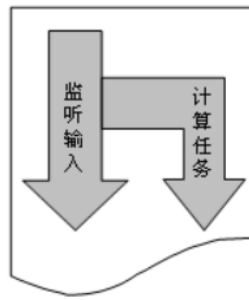
- 返回值

- 成功返回0，出错时返回错误码



训练原理：线程的设计

- 多线程程序主要用于需要并发执行的场合
- 例如在游戏中，需要维持一个动画场景，而玩家可以通过鼠标或键盘来输入操作指令，控制游戏的进行。
- 假如程序只有一个控制流，则当程序执行到等待用户输入指令的时候，由于用户输入较慢（相对 CPU 速度来说），程序无法向前继续执行，因而无法更新动画，效果上动画就停顿了
- 在线视频播放也存在同样的问题



训练原理：线程的设计

- 线程的启动

- 线程创建后等待系统调度，被调度后从线程启动例程函数开始执行
- 一次创建多个线程，其调度顺序和创建顺序无关！！



- 可以在线程启动例程中设计业务流程，也可以在线程启动例程中调用其他函数
 - 循环（可以是无限循环，直到进程生命周期终止！）
 - 条件
 - 分支



训练原理：线程的设计

- 线程启动例程函数

`void * start_rtn(void *arg)`

- 参数

- `void *arg`，可以是指向任意数据结构的指针，包括整形、结构体、缓冲区等。此时该参数由 `pthread_create` 的第四个参数传入

- 不需要传递参数时可以变化为 `void * start_rtn(void)`，此时 `pthread_create` 的第四个参数为 `NULL`

- 返回值

- `void *`，可以返回指向任意数据结构的指针，包括整形、结构体、缓冲区等



- 线程的终止

- 线程从启动例程函数中返回，函数返回值作为线程的退出码
- 线程调用pthread_exit函数终止自身执行
- 线程被同一进程中的其他线程取消



训练原理：pthread_exit函数

- 线程终止函数

- 头文件：pthread.h

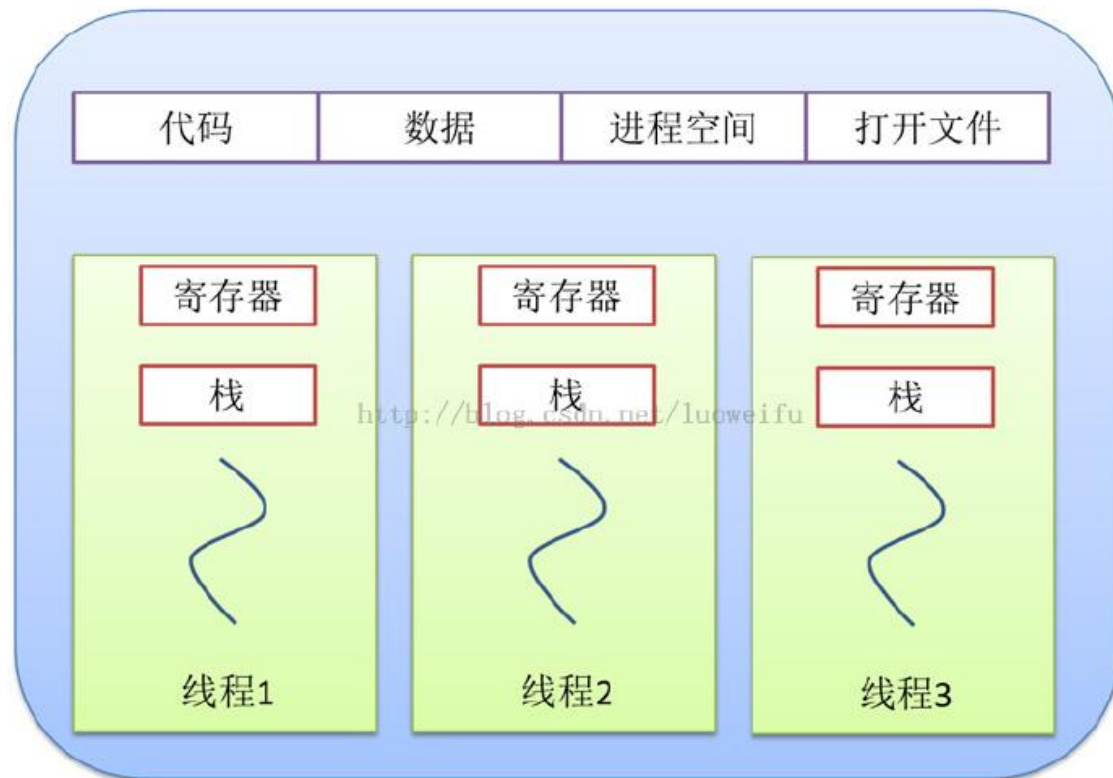
- **void pthread_exit(void *rval_ptr);**

- 参数

- rval_ptr：该指针将传递给等待线程终止函数（与exit函数参数用法类似）



训练原理：线程的数据共享



训练原理：线程的数据共享

- 一个进程内的所有线程共享进程的多种数据和资源（这种机制又带来了同步问题）
- 线程间共享的数据和资源：进程代码段、进程中的全局变量、进程打开的文件.....
- 每个线程私有的数据和资源：线程ID、线程上下文（一组寄存器值的集合）、线程局部变量（存储在栈中）



训练原理：父线程等待子线程终止

- 父线程等待子线程终止函数原型
 - 头文件：pthread.h
 - **int pthread_join(pthread_t thread, void **rval_ptr);**
- 调用该函数的父线程将一直被阻塞，直到指定的子线程（通过线程ID）终止



训练原理：父线程等待子线程终止

- 参数

- **thread**: 线程ID, 指定需要等待的子线程
- **rval_ptr**: 接收子线程终止返回值的指针 (若不关心该返回值, 可直接将该参数设置为空指针NULL)
 - ① 若线程从启动例程返回, rval_ptr将指向返回值
 - ② 若线程被其他线程取消, rval_ptr指向的值置为PTHREAD_CANCELED
 - ③ 若线程通过调用pthread_exit终止, rval_ptr就是pthread_exit函数中指定的参数

- 返回值

- 成功返回0, 出错返回错误编号



训练原理：创建并等待子线程代码示例

```
void *childthread(void *arg){  
    int i;  
    for(i=0;i<10;i++){  
        printf("childthread  
message:%s\n",(char *)arg);  
        sleep(1);}  
    return 0;}
```

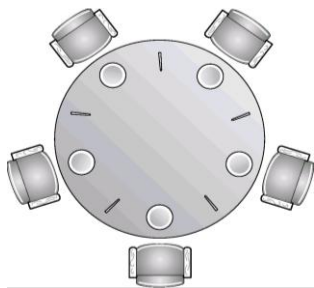
```
int main(int argc,char** argv){  
    pthread_t tid;  
    char str[100];  
    printf("create childthread\n");  
    sprintf(str,"str from parent");  
    pthread_create(&tid,NULL,(void *)  
childthread,str);  
    pthread_join(tid,NULL);  
    printf("childthread exit\n");  
    return 0;}
```



- 多个源文件编译为一个可执行文件
 - **gcc file1.c file2.c -o objfile.o**
 - 外部库的链接
 - 在Linux中线程控制API并未实现在标准C库中而是实现在了线程库中
 - 应用程序中使用了外部库需要在编译时进行显示链接
- gcc file1.c file2.c -o objfile.o -lpthread**

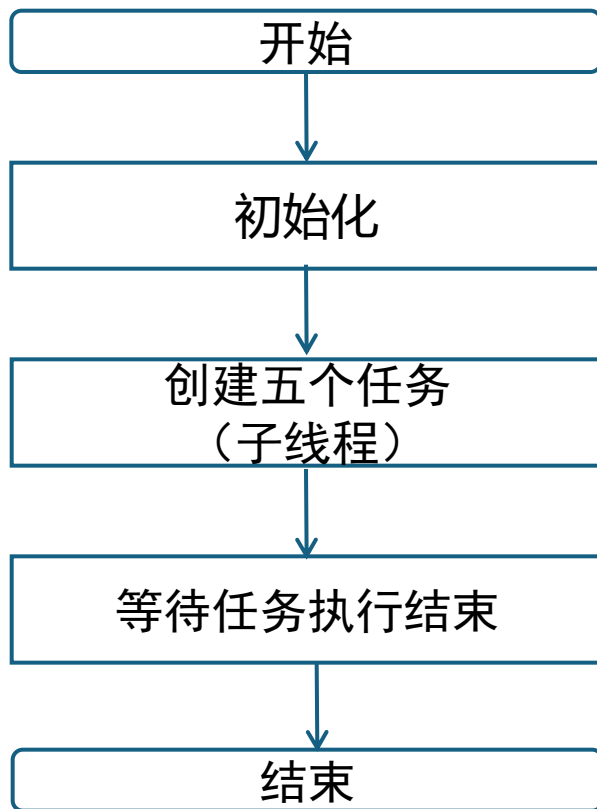


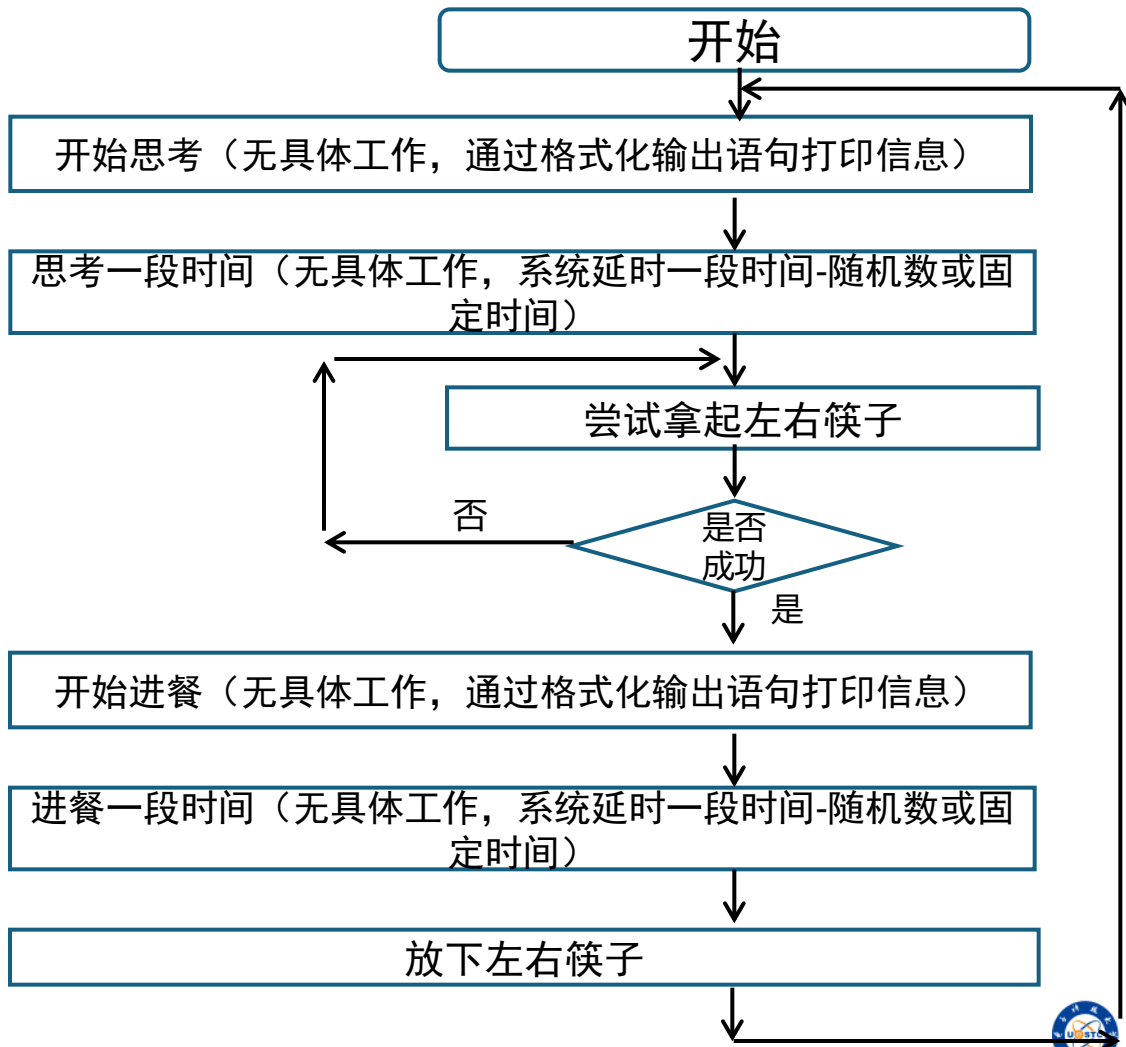
五个哲学家共用一张圆桌，分别坐在周围的五张椅子上，在圆桌上有五个碗和五只筷子，他们的生活方式是交替地进行思考和进餐。平时，一个哲学家进行思考，饥饿时便试图取其左右最靠近它的筷子，只有他拿到两只筷子时才能进餐。进餐毕，放下筷子继续思考。



训练原理：哲学家进餐问题的程序实现

main函数





哲学家任务



训练原理：哲学家进餐问题的程序实现

```
void philosopher(int number) {  
    while(1){  
        printf( "philosopher %d is thinking\n" ,number);  
        sleep(2);  
        takechopstick(number);  
        takechopstick((number+1)%5);  
        printf( "philosopher %d is eating\n" ,number);  
        sleep(2);  
        putchopstick(number);  
        putchopstick((number+1)%5);}}
```

代码示例：哲学家任务用线程来实现，对哲学家和筷子进行编号，并通过参数来传递编号



训练原理：拿起放下筷子操作

筷子是临界资源，拿起筷子操作就哲学家（不同任务）之间是对临界资源的互斥访问，放下筷子就是对临界资源的释放，在Linux中可以通过多种机制来实现

- **互斥量（加锁，解锁）**
- **XSI信号量集（P，V操作）**
- **POSIX信号量（P，V操作）**

训练原理：互斥量的操作

- ① 定义一个互斥量变量（`pthread_mutex_t` 类型）
- ② 初始化互斥量
- ③ 访问临界资源前对互斥量加锁
- ④ 访问临界资源后对互斥量解锁
- ⑤ 销毁互斥量

训练原理：互斥量初始化

- 头文件：pthread.h

①定义和静态初始化互斥量变量

pthread_mutex_t mlock = PTHREAD_MUTEX_INITIALIZER

②动态初始化

- 函数原型

int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);

- 参数与返回值
 - mutex: 互斥变量指针
 - attr: 设置互斥量的属性，通常采用默认属性，将attr设为NULL
 - 成功返回0，出错返回错误码

训练原理：互斥量销毁

- 互斥量在使用完毕后，必须要对互斥量进行销毁，以释放资源
- 函数原型（头文件：pthread.h）

int pthread_mutex_destroy(pthread_mutex_t *mutex);

- 参数与返回值
 - mutex：互斥量变量指针
 - 成功返回0，出错返回错误码

训练原理：互斥量的加锁和解锁操作

- 在对共享资源访问之前和访问之后，需要对互斥量进行加锁和解锁操作
- 函数原型（头文件：pthread.h）
`int pthread_mutex_lock(pthread_mutex_t *mutex);`
`int pthread_mutex_unlock(pthread_mutex_t *mutex);`
- 当调用pthread_mutex_lock时，若互斥量已被加锁，则调用线程将被阻塞
- 成功返回0，出错返回错误码

训练原理：非阻塞加锁

- 函数原型（头文件： pthread.h）

int pthread_mutex_trylock(pthread_mutex_t *mutex);

调用该函数时：

- 若互斥量未加锁，则锁住该互斥量，返回0；
- 若互斥量已加锁，则函数直接返回失败（不会阻塞调用线程）

POSIX信号量

- 建一个新的信号量或打开一个已存在的信号量

```
sem_t *sem_open(const char *name,int oflag,mode_t  
mode,unsigned int value);
```

- 删除信号量

```
int sem_unlink(const char *name);
```

- 信号量P操作

```
int sem_wait(sem_t *sem);
```

```
int sem_trywait(sem_t *sem);
```

- 信号量V操作

```
int sem_post(sem_t *sem);
```

训练

利用POSIX API在Linux系统上编写应用程序，通过多线程和互斥量机制/POSIX信号量机制实现哲学家进餐问题（**哲学家的数量可以通过简单的配置进行修改**）

- 首先通过阻塞加锁的操作方式实现哲学家之间（线程）对筷子（临界资源）的互斥访问，**观察程序运行一段时间以后会出现的死锁状态。**

如果未出现死锁状态的可以通过修改哲学家数量以及修改延时设置来增大出现死锁的机率。

- 将互斥量的加锁操作从阻塞方式修改为非阻塞方式，通过让权等待的思想预防死锁状态的出现。



Webserver V0.5版

多线程请求处理与资源竞争



mini_webserver

工程基础模块 Day1 && Day4

构建子模块 D1
Makefile
编译、清理、调试版本

配置子模块 D1
Config.c
端口、根目录、线程数

日志子模块 D1,D4
log.c
访问日志、错误日志

用户数据模块 Day2 && Day3

用户结构子模块 D2-D3
user.c
链表、二叉树、增删改查

CSV存储子模块 D2
csv_store.c
用户数据加载和保存

索引查找子模块 D3
user_index.c
按用户名快速查找

网络服务模块 Day6 && Day7

TCP监听子模块 D6
server.c
socket, bind, listen

连接管理子模块 D7
client.c
连接与状态、关闭释放

并发处理模块 Day4-5 && Day8-9

进程模型子模块 D4
process.c
父子进程协同

线程同步子模块 D5
thread.c
锁、条件变量、共享资源保护

线程池子模块 D8
thread_pool.c
任务队列、工作线程

事件驱动子模块 D9
event_loop.c
select / epoll

HTTP业务模块 Day10-Day14

请求解析子模块 D10
http_request.c
方法、URL、Header、Body

静态资源子模块 D11
static_file.c
HTML、CSS、图片读取

GET/POST处理子模块 D12
handler.c
表单提交、参数读取

路由配置子模块 D13
router.c
URL匹配、处理函数绑定

认证安全子模块 D14
auth.c
登录校验、权限控制、路径检查

响应生成子模块 D10-D14
http_response.c
状态码、响应头、响应体

测试模块 Day1-15

每日增加功能的相关测试

如: test_day01.sh

如: bench.c 并发压力测试/响应时间等

Web Server 后面要同时处理多个客户端请求。V0.4版本用请求文件模拟客户端请求。父进程负责扫描请求文件并创建子进程，子进程负责处理一个请求，父进程等待所有子进程结束并记录结果。

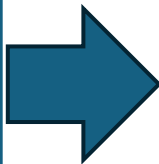
- 进程间有隔离，资源开销更大；
- 而线程共享内存，通信方便，但更容易出现数据竞争
- V0.5版本改成多线程 worker，做共享请求队列。

V0.5版 mini_webserver

requests/hello.req
GET /hello

requests/user_find.req
GET /users/zhangsan

requests/missing.req
GET /not-found



outputs/hello.out
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 12
Hello, Web!

outputs/user_find.out
FOUND zhangsan 123456 13800000000

outputs/missing.out
HTTP/1.1 404 Not Found



用请求文件模拟客户端请求。父进程负责扫描请求目录，把请求加入队列；多个子进程从队列中取任务并处理。

父线程

- 父进程先扫描 requests/
- 父进程把任务路径放到 request_queue
- 父进程生成**多个** worker线程
- 每个 worker都从队列取任务

子线程

根据信号量决定是否去共享队列获取
获取请求后处理请求
输出结果到 outputs/
继续根据信号量获取新请求



V0.5版 mini_webserver-检查内容

- 主线程扫描 requests/
- 主线程把请求路径放入共享队列
- 创建指定数量 worker 线程
- worker 线程从队列取任务
- 每个请求只处理一次
- 输出结果写入 outputs/
- 用**互斥量**保护请求队列
- 用**条件变量**等待任务
- 用互斥量保护统计数据
- 主线程用 pthread_join 等待所有 worker 结束

